

Document 02 — Technical Requirements

Technical Requirements Document

Document 02 — Technical Requirements

System overview

VelaDial uses two ESPHome devices connected to Home Assistant over WiFi. Home Assistant owns the lighting entities and communicates with the Tuya bulbs locally. The ESPHome devices act as input surfaces and lightweight displays; they do not own long-term state.

Local-first, not Home Assistant-independent

VelaDial is **local-first**, not **Home Assistant-independent**. The normal control path is:

```
ESPHome device → local WiFi / LAN → Home Assistant on Raspberry Pi → LocalTuya / local LAN → bedroom
```

The relevant outage types are different and should not be lumped together.

Internet / ISP outage. Normal light control does **not** require the internet. LocalTuya talks to the Tuya bulbs over the LAN, ESPHome talks to Home Assistant over the local API, and the ESP32 nodes talk to the Pi over the LAN. If the household internet or ISP goes down but the local WiFi / LAN remains healthy, VelaDial should continue to work normally. **Internet outage alone is not a blocker for normal control.**

WiFi / router / AP / LAN outage or instability. This is the **real network risk** for VelaDial. If the ESP32 devices, the Raspberry Pi, or the Tuya bulbs drop off the local WiFi / LAN (router reboot, AP flapping, WPA renegotiation issues, DHCP churn, RF interference), commands may not reach the bulbs even though the internet is fine. Local-first does not protect against LAN-side failure.

Home Assistant / Raspberry Pi failure. If Home Assistant itself is unavailable (HA crashed, container restarting, Pi rebooting, OS hung), ESPHome actions that depend on Home Assistant may not reach the bulbs.

ESP32-side inputs. Any ESP32-side input (touch, knob, gesture, or optional button) still requires the WiFi + Home Assistant + LocalTuya path to actually change the bulbs unless a separate direct ESP32-to-bulb control path is implemented. Such a direct path is **not** part of the first build, so **ESP32 buttons or LEDs alone should not be described as a true WiFi / HA outage fallback.**

Network reliability recommendations

These are deployment recommendations to keep the LAN reliable. They reduce the likelihood of WiFi / router / AP / LAN outages, which are the dominant network-side risk for VelaDial.

- Connect the Raspberry Pi to the router via Ethernet if possible. Keeps the Pi off the WiFi airwaves and removes one class of flake.
- Put the Raspberry Pi and the router / WiFi AP / network gear on a UPS if possible. A Pi-only UPS helps only if the network gear remains powered through the same blip; the LAN path must stay end-to-end for control to survive.
- Use a stable 2.4 GHz IoT SSID for the Tuya bulbs and ESP32 devices. Both ESP32-S3 and ESP32-C6 use 2.4 GHz WiFi; Tuya bulbs are typically 2.4 GHz only.
- Avoid band steering / Smart Connect on the IoT SSID if it causes instability. IoT devices and dual-band APs do not always negotiate well.
- Use WPA2 2.4 GHz compatibility mode if needed for older Tuya bulb firmware. WPA3-only or mixed WPA2/WPA3 with PMF-required can confuse some IoT devices.
- Reserve static DHCP leases in the router (to avoid IP churn that breaks LocalTuya and ESPHome connections) for:
 - Raspberry Pi / Home Assistant
 - door-side ESP32-S3
 - bedside ESP32-C6
 - each Tuya bulb
- Keep control local through Home Assistant + LocalTuya / local LAN. Do not add cloud dependency for normal control.

Optional future network improvements

These are future / advanced options, **not first-build requirements**. Pursue only if testing shows the household WiFi is unreliable in the actual VelaDial deployment.

- **Dedicated IoT access point** for the bulbs + ESP32 devices, separate from the household WiFi. Often a cheap second AP wired into the same LAN.
- **Pi-hosted 2.4 GHz AP** (Pi running an AP daemon such as `hostapd` + `dnsmasq` for the IoT devices). Advanced setup, more Pi load, only worth it if a dedicated AP is not an option.
- **Pi-side fallback daemon** for Home Assistant software failure (a small service on the Pi using a local Tuya library to talk to bulbs directly when HA is down) can be researched later. It does **not** solve Pi power loss, network failure, or WiFi outage, and adds a second control path that must be coordinated with HA.
- **Matter / Thread or Zigbee bulb migration** (ESP32-C6 has an 802.15.4 radio; bulbs would need replacement). Future v2 / v3 architecture change, not first build.

Manual wall switch as true everything-broken fallback

A manual wall switch on the bulb circuit is the only simple **true "everything is broken" fallback**. It works regardless of which part of the stack has failed — Raspberry Pi, ESP32, router, WiFi AP, or Home Assistant — because it cuts mains power to the bulbs directly.

It is a separate concern from the network reliability recommendations above. The recommendations reduce the likelihood of partial-failure scenarios; the wall switch covers total-failure scenarios.

Relay or mains-control hardware on the ESP32 is **not** part of the first-build firmware. The wall switch lives at the wall, not in the firmware.

Hardware

Door-side rotary display (ESP32-S3)

- ELECROW CrowPanel 1.28 in rotary display.
- ESP32-S3 class board.
- GC9A01A round IPS display, 240 × 240.
- Rotary encoder and capacitive touch input.
- Optional WS2812 LED ring.
- TSL2591 ambient light sensor, I2C address `0x29`. Purpose: ambient lux sensing for adaptive display backlight and day/night UI behavior.
- SHT45 temperature/humidity sensor, I2C address `0x44`. Purpose: room temperature and humidity diagnostics, secondary/future UI only.

ELECROW board revision warning. Multiple ELECROW CrowPanel 1.28 in revisions may exist with subtly different GPIO assignments. The pinout documented in [hardware/elecrow_pinout.md](#) must be verified against the actual board silkscreen / PCB revision before any production firmware is written. Run an I2C scan and confirm display, touch, encoder, and WS2812 pins with a minimal bring-up YAML against real hardware before relying on any GPIO assignment.

Bedside gesture controller (ESP32-C6)

- Adafruit ESP32-C6 Feather or equivalent ESP32-C6 board.
- APDS-9960 gesture/proximity sensor over I2C, address `0x39`. Purpose: directional gestures, used for left/right gesture control.
- VL53L4CD time-of-flight distance sensor over I2C, default address `0x29`. Purpose: reliable hand-near / hold detection for nightlight mode.
- Small bedside enclosure with the sensors exposed or behind a suitable window.

Optional bedside hardware improvements (first build, not required)

These are optional first-build additions, not required architecture changes:

- **Status LED** (any small LED + current-limiting resistor on a free GPIO): provides silent local feedback that a bedside gesture or button was detected by the ESP32. Lights up before the Home Assistant round-trip completes, so the user can tell the input was registered even if bulbs respond slowly.
- **Push button** (any momentary tactile switch on a free GPIO): a reliable physical input when gestures misfire (sunlight interference, awkward hand position). Fires a normal `homeassistant.action` like any other ESPHome binary sensor.

Important framing:

- The status LED is useful as silent feedback.
- The push button is useful as a reliable physical input.
- **Neither is a Home Assistant / WiFi outage fallback by itself.** Both still depend on the ESPHome → Home Assistant → LocalTuya path to actually change the bulbs. A true outage fallback would require a separate direct bulb-control path on the ESP32 (for example, a community-maintained `tuya_local` component) or a manual wall switch, and neither is in scope for the first build.

Hub

- Home Assistant OS on Raspberry Pi 4/5 or another always-on local host.
- ESPHome add-on or ESPHome CLI for firmware build and OTA updates.
- HACS if LocalTuya is used.

I2C architecture

The door-side node and bedside node are separate ESP32 devices with separate I2C buses.

Door-side I2C bus (ESP32-S3)

SENSOR	I2C ADDRESS	ESPHOME PLATFORM
TSL2591	<code>0x29</code>	<code>tsl2591</code> (native)
SHT45	<code>0x44</code>	<code>sht4x</code> (native)

Bedside I2C bus (ESP32-C6)

SENSOR	I2C ADDRESS	ESPHOME PLATFORM
APDS-9960	0x39	apds9960 (native)
VL53L4CD	0x29	See note below

TSL2591 and VL53L4CD both use 0x29, but this is not a conflict in the first-build architecture because they are on different ESP32 nodes. XSHUT/address reassignment or a TCA9548A multiplexer is only a future fallback if both 0x29 sensors are ever placed on the same I2C bus.

All sensors connect via STEMMA QT / Qwiic connectors for solderless daisy-chaining within each node.

Software stack

- ESPHome for firmware configuration and device integration.
- Home Assistant as the automation/control hub.
- LocalTuya or another local Tuya LAN integration for RGB bulb control.
- LVGL inside ESPHome for the round display UI.
- Multi-theme firmware engine (20 selectable UI themes, switchable on-device via hidden Theme Selector).

ESPHome support notes

- **TSL2591:** Expected supported via the tsl2591 platform. Auto-gain recommended for bedroom use (handles both direct sunlight and pitch-dark conditions).
- **SHT45:** Expected supported via the sht4x platform.
- **APDS-9960:** Expected supported via the apds9960 platform. Requires tuning of gesture sensitivity and proximity thresholds.
- **VL53L4CD:** ESPHome support is **unverified**. Before writing production firmware, verify against current official ESPHome documentation or a trusted component source whether a native ESPHome platform, a maintained community component, or a custom component is available.
- **VL53L0X (verified fallback only):** Currently the **verified** ESPHome-supported ToF fallback if VL53L4CD support is blocked. Range and minimum-distance characteristics differ from VL53L4CD (VL53L0X has a higher minimum range), so hold-detection thresholds must be re-tuned.
- **VL53L1X:** Not assumed native. Do not treat VL53L1X as a verified ESPHome fallback unless current official ESPHome documentation or a trusted component source confirms it.

Sensor timing and power notes

- **TSL2591 and SHT45** can update on the order of seconds without harming the user experience. They feed display backlight and diagnostics, neither of which needs fast polling.
- **APDS-9960 gesture detection** requires polling or event handling fast enough to catch a swipe (typically sub-second). Update intervals on the order of tens of seconds will cause gestures to be missed entirely. Avoid configurations that under-sample the gesture sensor.
- **ToF distance readings** for hand-hold detection need stable filtering (for example, moving-average or hysteresis) and a hold-time threshold (around 1.5 s) combined with a cooldown after each trigger. Raw distance values fluctuate and must not directly drive bulb actions.
- **Avoid sensor logic that can trigger full-brightness room light changes from noise.** Any path from a sensor reading to a bulb command must be debounced, edge-triggered, and cooldown-gated. A bedroom is a worst-case environment for accidental full-brightness flashes at night.
- **Power and current draw** should be measured during bring-up, not estimated. Both VL53L4CD and APDS-9960 use active IR emitters, and the ESP32-C6 with WiFi active is non-trivial draw. No battery design decisions should be made until real current measurements are taken on the assembled hardware.

Home Assistant entities

Expected entities should stay configurable, but the first build assumes:

- `light.bedroom_group` — group containing the five RGB bulbs.
- Individual Tuya bulb entities may be named differently in a real installation.
- `sensor.room_ambient_light` — TSL2591 lux reading (door-side node).
- `sensor.room_temperature` — SHT45 temperature (door-side node).
- `sensor.room_humidity` — SHT45 humidity (door-side node).
- `sensor.bedside_distance` — VL53L4CD distance reading in mm (bedside node).

Security and local configuration

Live network credentials, OTA credentials, API credentials, and Tuya local keys must stay outside the repository in the local ESPHome/Home Assistant configuration. The example firmware expects those values to be supplied by the local ESPHome secrets file.

Functional requirements

- Left gesture turns the light group off.

- Right gesture turns the light group on.
- Proximity hold (VL53L4CD: hand steady at 5–10 cm for 1.5 s) triggers a warm 10% nightlight mode.
- Rotary encoder updates brightness in predictable increments.
- Touch UI supports power and preset selection.
- Hidden Theme Selector (long-press >1.5s) allows switching between 20 UI themes without reflashing.
- Active theme persists across reboots and is exposed as a diagnostic text sensor to Home Assistant.
- Display sleeps or dims after inactivity.
- Display backlight adapts automatically to ambient room light (TSL2591 lux → backlight PWM).

Non-functional requirements

- Local LAN operation for normal use.
- No mechanical relay in the lighting control path.
- Fast response target: under 500 ms from user action to visible light change.
- Safe defaults after reboot.
- YAML should remain readable enough for manual debugging.
- Multi-theme engine must fit within ESP32-S3 RAM/Flash constraints (target: <25% RAM, <80% Flash).

Known implementation notes

- Pin mappings for the ELECROW display are tracked in [hardware/elecrow_pinout.md](#). Pinout must be validated against the actual PCB revision; see the board revision warning under Door-side hardware.
- Display driver, touch transform, and encoder direction should be verified on real hardware.
- LocalTuya entity IDs and datapoints may vary by bulb model and firmware.
- VL53L4CD ESPHome integration status must be confirmed during firmware development. Verified ToF fallback: **VL53L0X** (native ESPHome support; re-tune thresholds for its higher minimum range). VL53L1X support is unverified and should not be assumed without current official ESPHome documentation or a trusted component source. A custom C++ component remains an option if no native or community-maintained component fits.